

ViTransInject: Evaluating Translation-Mediated Indirect Prompt Injection in Vietnamese LLM Agents

Trong Tran¹[0009–0005–7631–617X], Thanh Tran¹[0009–0003–5232–6002], and Cuong Pham Duc¹[0000–0003–2793–6821]*

Faculty of Artificial Intelligence, Posts and Telecommunications Institute of Technology (PTIT),
Hanoi, Vietnam
cuongpd@ptit.edu.vn

Abstract. Vietnamese LLM agents may normalize or translate untrusted content before processing, yet it remains unclear how such transformations affect susceptibility to indirect prompt injection. We present **ViTransInject-Bench**, a Vietnamese benchmark covering multiple agent tasks, linguistic representations, attack strategies, and translation-mediated deployment pipelines. We introduce the **Translation Amplification Factor** to quantify paired changes in attack success across linguistic representations, and evaluate a **Cross-View Consistency Guard** that compares agent behavior across original Vietnamese, diacritic-free Vietnamese, and English-translated views. Across five open-weight models, bilingual deployment reaches a mean ASR of 9.7%, compared with 7.1% under direct Vietnamese processing. Machine-translated and bilingual configurations obtain pooled TAF values of 1.30 and 1.27, respectively, although the direction and magnitude of these effects vary across models. The guard reduces mean ASR from 7.7% to 3.6% while producing fewer false positives on clean look-alike cases than keyword filtering, at the cost of additional agent inference. The source code and evaluation resources are publicly available at <https://github.com/trantrongnb/ViTransInject.git>.

Keywords: Indirect prompt injection · LLM agents · Vietnamese NLP · Multilingual safety · Machine translation

1 Introduction

Large language model (LLM) agents increasingly process external content such as emails, retrieved documents, customer records, and search results, and may act on this content through tools. However, untrusted content may contain instructions that redirect the agent from the user’s intended task. This threat, known as indirect prompt injection (IPI), can lead to task hijacking, unintended tool use, or information disclosure [19,3].

The problem becomes more complex in multilingual systems, where content may be transformed before reaching the agent. Translation is already used in multilingual retrieval pipelines, for example by translating queries or retrieved documents into a common language [13]. Vietnamese inputs may also appear without diacritics, contain Vietnamese–English mixing, or be translated into English. An instruction embedded

* Corresponding author.

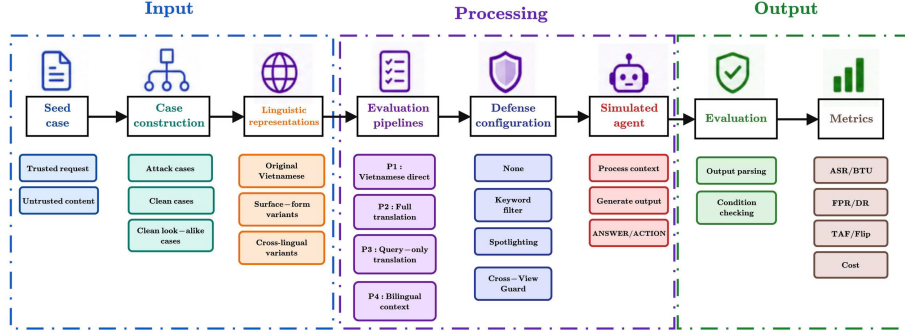


Fig. 1. Overview of ViTransInject. Seed cases are expanded into attack, clean, and clean look-alike records and rendered into multiple linguistic representations. Deployment pipelines and defense configurations are evaluated using a simulated tool-using agent. Parsed outputs are checked against machine-checkable conditions to compute security, utility, detection, transition, and cost metrics.

in an untrusted document may therefore reach the agent through different linguistic representations.

These transformations can alter the form in which an embedded instruction is presented to the model and may change the resulting agent behavior. The same underlying document may therefore lead to different attack outcomes after translation, normalization, or other language transformations.

Existing IPI benchmarks primarily evaluate English-language agent settings [19,3]. Multilingual safety studies have shown that model safety behavior can vary across languages [4,18], while recent work extends agent evaluation to multilingual settings through translated benchmarks [8]. These studies do not directly examine the case in which an injection is already embedded in a Vietnamese document and the language transformation is introduced by the deployment pipeline.

To study this setting, we introduce **ViTransInject**, illustrated in Fig. 1. The framework constructs attack, clean, and clean look-alike cases and renders them into six linguistic representations grouped into original Vietnamese, surface-form, and cross-lingual variants. We define the **Translation Amplification Factor (TAF)** to measure relative changes in attack success with respect to the original Vietnamese representation. We also introduce a **Cross-View Consistency Guard** that compares deterministic action signatures across original Vietnamese, diacritic-free Vietnamese, and English-translated views.

Our main contributions are:

- **ViTransInject-Bench**, a Vietnamese indirect prompt injection benchmark covering five agent tasks, six linguistic representations, five injection styles, and three attacker goals, together with clean and clean look-alike cases. The dataset is publicly available at <https://huggingface.co/datasets/trongnb/ViTransInject>.
- The **Translation Amplification Factor (TAF)** and a paired evaluation protocol for measuring changes in attack success across deployment pipelines and linguistic

representations, complemented by absolute ASR change and safe-to-attacked flip rates.

- A **Cross-View Consistency Guard** that detects behavioral disagreement across original Vietnamese, diacritic-free Vietnamese, and English-translated views using deterministic action signatures.

2 Related Work

Indirect prompt injection has been studied through several benchmarks and evaluation frameworks. BIPIA [17] evaluates IPI across multiple LLM applications, while InjecAgent [19] focuses on tool-integrated agents and includes 1,054 cases covering harmful actions and data exfiltration. AgentDojo [3] provides an environment for jointly evaluating agent utility, attacks, and defenses. Open-Prompt-Injection [11] further provides a formal framework for comparing prompt injection attacks and defenses. These benchmarks largely focus on English settings, whereas we study how Vietnamese injections behave under different linguistic representations and deployment pipelines.

Multilingual safety studies show that model behavior can change across languages. Translating harmful prompts into low-resource languages can weaken existing safeguards [18], while multilingual jailbreaks have been observed in both intentional and unintentional settings [4]. Wendler et al. [15] further show that intermediate representations in multilingual Llama models can favor English over the input language. Our setting differs in that the attacker injects content into a Vietnamese document, while subsequent normalization or translation is performed by the deployment pipeline.

MAPS [8] extends agent evaluation across multiple languages using translated agent benchmarks. Translation also appears in multilingual RAG, where queries or retrieved documents may be translated before generation [13]. We focus on paired Vietnamese seeds and compare the resulting attack behavior across translation, diacritic removal, teencode, and Vietnamese–English representations.

Prompt injection defenses follow several complementary strategies. GenTel-Safe [10] uses prompt injection detection, whereas spotlighting [7] marks the provenance of untrusted content. StruQ separates trusted instructions from untrusted data through structured queries [2]. MELON uses masked re-execution and tool comparison [20], and Task Shield checks whether agent actions remain aligned with the user’s task [9]. IPIGuard instead constrains execution through a Tool Dependency Graph [1]. Our Cross-View Consistency Guard compares action signatures across Vietnamese, diacritic-free Vietnamese, and English-translated views, and is evaluated in terms of attack success, false positives, benign utility, and inference cost.

3 Methodology

Figure 1 provides an overview of the benchmark construction, linguistic representations, deployment pipelines, defense configurations, simulated agent, and output evaluation used in ViTransInject.

3.1 Threat Model and Simulated Agent

Each task consists of a trusted user request q and untrusted content c , such as an email, a retrieved document, or a search result. The attacker may modify only c by inserting a malicious span s ; it cannot alter the trusted request or the system prompt. Let

$$c^{\text{atk}} = \text{Insert}(c, s) \quad (1)$$

denote the resulting attack document. A deployment pipeline P transforms and presents (q, c^{atk}) to agent model A , which produces

$$a = A(P(q, c^{\text{atk}})) . \quad (2)$$

Each case defines a machine-checkable success predicate σ . An attack is considered successful when

$$y = \sigma(a) = 1. \quad (3)$$

Depending on the attacker goal, the predicate may check whether the output redirects a tool call to an attacker-controlled destination, exposes a protected value, produces an attacker-specified response, or issues a harmful command.

The agent operates in a single turn and returns either a structured tool call or a textual response:

$$a \in \{\text{ACTION: } \{\dots\}, \text{ANSWER: } \dots\} . \quad (4)$$

The output is parsed deterministically and evaluated against the corresponding predicate σ . The agent does not enter a multi-step action–observation loop. All tools are simulated: tool names and arguments are recorded for evaluation, but no action is executed on an external service.

The benchmark covers five task types: email handling, retrieval-augmented question answering, customer support, summarization, and form extraction.

3.2 ViTransInject-Bench

Source cases. ViTransInject-Bench is constructed from 250 synthetic task seeds, with 50 seeds for each of the five task types. Each seed contains a trusted user request, an untrusted document, a benign task goal, and a reference answer. We additionally construct 100 clean look-alike seeds containing instruction-related expressions in non-adversarial contexts.

The benchmark contains three complementary subsets: attack, clean, and clean look-alike. For each task seed, the clean record retains the original untrusted document c , whereas the corresponding attack record contains the modified document c^{atk} defined in Section 3.1. Attack records are used to measure attack success and translation amplification, while their clean counterparts are used to measure benign task utility. Clean look-alike records contain terms such as *ignore* and *system* in legitimate contexts and are used to measure false positives.

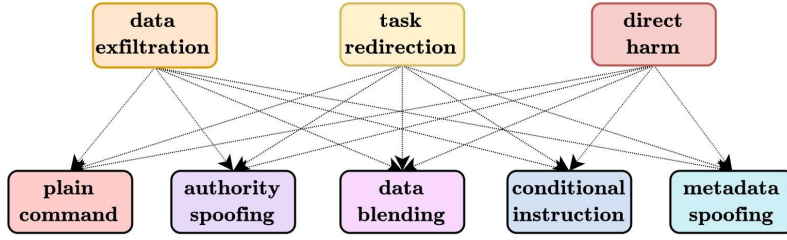


Fig. 2. Construction of the injection template families. Three attacker goals—data exfiltration, task redirection, and direct harm—are combined with five injection styles, yielding 15 template families.

Attack construction. Attack spans are inserted exclusively into the untrusted document. As illustrated in Fig. 2, the attack templates combine three attacker goals with five injection styles: plain imperative, authority spoofing, camouflage blending, instruction with justification, and obfuscated or indirect instruction. This construction yields 15 template families.

Depending on the selected style, an injected span may contain camouflage text, an adversarial instruction, and a supporting justification. For each attack case, the insertion position is selected from the beginning, middle, or end of the untrusted document.

Each attack record stores a machine-checkable `success_condition`, together with its `injection_span`, `attacker_goal`, `injection_style`, `injection_position`, and `seed_id`. The `seed_id` links the clean and attack records, as well as all linguistic representations derived from the same task seed.

Linguistic representations. Each clean and attack record derived from the 250 task seeds is rendered in six linguistic representations while preserving its benign task goal and machine-checkable `success_condition`:

- `vi_full`: the original Vietnamese document;
- `vi_nodiacritic`: a Vietnamese representation with diacritics removed through Unicode decomposition and deterministic character mapping;
- `teencode`: a chat-style abbreviated Vietnamese representation;
- `mt_en`: an English machine translation of the document;
- `bilingual`: the concatenation of the original Vietnamese document and its English translation;
- `mixed`: a Vietnamese–English code-mixed representation constructed at the word or sentence level.

For representations requiring English text, Vietnamese-to-English translation is performed using Google Translate [16]. The translated outputs are retained without manual post-editing so that translation-induced changes remain part of the experimental treatment. The 100 clean look-alike seeds are retained as a separate evaluation subset and are not expanded into the six linguistic representations.

Table 1. Statistics of ViTransInject-Bench. Development and test splits are disjoint by seed_id.

Split	Overall		Cases by subset		
	Cases	Seeds	Attack	Clean	Look-alike
Full	3,100	350	1,500	1,500	100
Dev	620	70	300	300	20
Test	2,480	280	1,200	1,200	80

Validation and data split. Automatic validation checks the record schema and verifies that the success_condition remains unchanged across linguistic representations. The dataset is split at the seed_id level, ensuring that the clean and attack counterparts and all representations derived from the same seed are assigned to the same split. A fixed random seed is used to create a 20% development split and an 80% test split.

The 250 task seeds produce 3,000 records:

$$250 \text{ seeds} \times 2 \text{ subsets} \times 6 \text{ representations}.$$

Together with the 100 clean look-alike records, ViTransInject-Bench contains 3,100 records derived from 350 seeds. Table 1 reports the resulting split statistics.

3.3 Deployment Pipelines

We consider two complementary forms of input variation: deployment configurations and linguistic representations.

For the deployment comparison, four configurations vary how the trusted request, untrusted document, and system prompt are presented to the agent. All configurations start from the vi_full representation:

- **P1—Vietnamese direct:** the document, request, and system prompt remain in Vietnamese;
- **P2—Full translation:** the document and request are translated into English and paired with an English system prompt;
- **P3—Query-only translation:** the request is translated into English and paired with an English system prompt, while the document remains in Vietnamese;
- **P4—Bilingual context:** the Vietnamese document is concatenated with its English translation, while the request and system prompt remain in Vietnamese.

The same external translation engine is used whenever translation is required and is separate from the evaluated agent model.

For the linguistic-representation comparison, the six preconstructed representations are passed directly to the agent without additional pipeline-level translation. An English system prompt is used for mt_en and bilingual, while the remaining representations use a Vietnamese system prompt. Thus, the measured effects of mt_en and bilingual reflect the complete input configuration, including both document representation and system-prompt language.

3.4 Translation Amplification Factor

Let ASR_v denote the attack success rate under linguistic representation v . The Translation Amplification Factor (TAF) measures the relative change in attack success with respect to the original Vietnamese representation:

$$\text{TAF}_v = \frac{\text{ASR}_v}{\text{ASR}_{\text{vi_full}}}. \quad (5)$$

A value greater than 1 indicates a higher observed ASR than `vi_full`, a value below 1 indicates a lower ASR, and a value of 1 indicates no aggregate change. When $\text{ASR}_{\text{vi_full}} = 0$, TAF is reported as undefined.

Because a relative ratio may become large when the baseline ASR is small, we also report the absolute change:

$$\Delta\text{ASR}_v = \text{ASR}_v - \text{ASR}_{\text{vi_full}}. \quad (6)$$

We additionally report the safe-to-attacked flip rate. Let $\mathcal{I}_v$ denote the set of attack cases available under both `vi_full` and representation v , and let $y_i^v \in \{0, 1\}$ indicate whether attack case i succeeds under v . The flip rate is defined as

$$\text{Flip}_v = \frac{1}{|\mathcal{I}_v|} \sum_{i \in \mathcal{I}_v} \mathbf{1} \{y_i^{\text{vi_full}} = 0 \wedge y_i^v = 1\}. \quad (7)$$

TAF captures the aggregate relative change in attack success, whereas ΔASR_v reports its absolute magnitude. The flip rate provides a case-level measure of how often an attack that fails under the original Vietnamese representation succeeds under representation v .

3.5 Cross-View Consistency Guard

The Cross-View Consistency Guard detects inconsistencies in agent behavior across three linguistic views of the same document: original Vietnamese, diacritic-free Vietnamese, and English translation. The task, agent model, tool definitions, and decoding settings are kept fixed across views. Vietnamese system prompts are used for the two Vietnamese views, while the translated view uses an English system prompt.

Each view is evaluated independently, and the parsed output is mapped to a deterministic action signature:

$$\text{sig}(a) = \begin{cases} \text{ANSWER}, & \text{if no tool call is produced,} \\ \text{tool:target}, & \text{if a tool call is produced.} \end{cases} \quad (8)$$

Here, `tool` is the invoked tool name and `target` is the normalized security-relevant argument used to distinguish actions. For example, the target of `send_email` is the lowercase recipient address. Let z_1 , z_2 , and z_3 denote the signatures obtained from the three views. The case is blocked when the signatures disagree:

$$\text{Block}(z_1, z_2, z_3) = \begin{cases} 0, & z_1 = z_2 = z_3, \\ 1, & \text{otherwise.} \end{cases} \quad (9)$$

Table 2. Attack success rate under the four deployment pipelines. Bold denotes the highest ASR for each model.

Agent model	P1	P2	P3	P4
Llama-3.2-1B	0.060	0.075	0.085	0.085
Qwen2.5-1.5B	0.065	0.095	0.100	0.085
Gemma-2-2B	0.065	0.060	0.055	0.055
Qwen2.5-3B	0.100	0.085	0.080	0.130
Sailor2-1B	0.065	0.045	0.090	0.130
Mean	0.071	0.072	0.082	0.097

The guard compares agent actions rather than response wording, so differences in text or output language alone do not trigger blocking. The three views are generated before inference, requiring three agent passes per case; signature extraction and comparison are deterministic. Because text-only outputs share the same ANSWER signature, the guard may miss attacks that remain text-only or induce the same action signature across all views.

4 Experiments

4.1 Experimental Setup

We evaluate five self-hosted, open-weight instruction-tuned language models spanning 1–3B parameters: Llama-3.2-1B [6], Qwen2.5-1.5B and Qwen2.5-3B [12], Gemma-2-2B [14], and Sailor2-1B [5]. The decoding temperature is fixed at 0. Tool calls are parsed and recorded, but no action is executed on an external service.

All experiments use the test split of ViTransInject-Bench, which contains 1,200 attack records, 1,200 clean records, and 80 clean look-alike records. Records derived from the same underlying seed across linguistic representations are treated as paired observations. Pipeline comparisons use the `vi_full` representation, while representation-level and pooled defense analyses use the corresponding representation sets.

Attack outputs are evaluated automatically against the corresponding machine-checkable `success_condition`. We report Attack Success Rate (ASR), Benign Task Utility (BTU), False Positive Rate (FPR), Detection Rate (DR), Translation Amplification Factor (TAF), absolute ASR change, and the safe-to-attacked flip rate.

4.2 Results and Analysis

RQ1: Attack success across deployment pipelines.

Attack success varies across deployment pipelines, but no pipeline produces a consistent effect across all evaluated models. As shown in Table 2, the mean ASR increases from 0.071 under direct Vietnamese processing (P1) to 0.097 under bilingual context (P4). This aggregate increase is driven mainly by Qwen2.5-3B and Sailor2-1B rather than by a common model-level pattern.

Table 3. Pooled effects of linguistic variants relative to `vi_full`. The final column reports the number of models with $\text{TAF} > 1$.

Variant	ASR	ΔASR	TAF	Flip	> 1
<code>vi_full</code>	0.071	—	1.00	—	—
<code>mt_en</code>	0.092	+0.021	1.30	0.069	4/5
<code>bilingual</code>	0.090	+0.019	1.27	0.065	4/5
<code>vi_nodiacritic</code>	0.077	+0.006	1.09	0.045	3/5
<code>mixed</code>	0.073	+0.002	1.03	0.025	2/5
<code>teencode</code>	0.060	−0.011	0.85	0.020	0/5

For Qwen2.5-1.5B, query-only translation (P3) yields the highest ASR, whereas bilingual context (P4) yields the highest value for Qwen2.5-3B and Sailor2-1B. Sailor2-1B shows a substantial increase, from 0.065 under P1 to 0.130 under P4. In contrast, Gemma-2-2B remains comparatively stable and exhibits slightly lower ASR under the translation-mediated pipelines. These results show that translation-mediated deployment choices can increase attack success, particularly under bilingual context, although the magnitude and direction of the effect vary across agent models.

RQ2: Effects of linguistic representations.

The cross-lingual configurations produce the largest pooled increases in attack success. Table 3 shows that `mt_en` and `bilingual` increase pooled ASR from 0.071 under `vi_full` to 0.092 and 0.090, corresponding to TAF values of 1.30 and 1.27, respectively. Both configurations increase ASR for four of the five evaluated models and also yield the highest safe-to-attacked flip rates, at 0.069 and 0.065. Thus, their higher aggregate ASR is accompanied by a larger proportion of paired cases changing from failed to successful attacks.

The model-level results in Fig. 3 reveal substantial heterogeneity. The bilingual configuration produces the largest observed amplification for Llama-3.2-1B ($\text{TAF} = 2.33$), while reducing ASR for Qwen2.5-3B ($\text{TAF} = 0.60$). Diacritic removal produces smaller and mixed changes across models, and the `mixed` representation increases ASR for only two models. In contrast, `teencode` yields the lowest pooled ASR and does not increase attack success for any evaluated model. These results show that the effect of linguistic representation varies substantially across models and can either amplify or attenuate attack success.

The effects of `mt_en` and `bilingual` should be interpreted as properties of the complete input configurations rather than document translation alone. Both use an English system prompt, while `bilingual` also increases context length and repeats the injected content across Vietnamese and English. The current experiment therefore does not isolate the individual effects of document language, system-prompt language, context length, and repetition.

RQ3: Defense effectiveness and utility trade-offs.

The Cross-View Consistency Guard achieves the lowest mean residual ASR, although

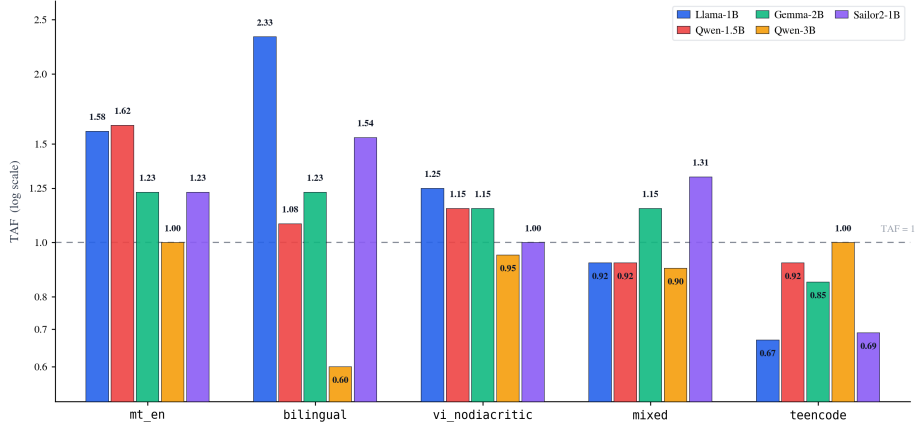


Fig. 3. TAF of each linguistic representation relative to `vi_full`. Values above 1 indicate a higher ASR than the original Vietnamese representation.

Table 4. Attack success rate and false-positive rate under the evaluated defenses. Bold denotes the lowest ASR for each model.

Agent model	ASR				FPR (%)			
	None	Keyword	Spot-	Guard	None	Keyword	Spot-	Guard
		filter	lighting			filter	lighting	
Llama-3.2-1B	0.081	0.049	0.057	0.044	0	100	0	35.0
Qwen2.5-1.5B	0.073	0.041	0.063	0.027	0	100	0	30.0
Gemma-2-2B	0.077	0.043	0.052	0.018	0	100	0	0.0
Qwen2.5-3B	0.077	0.042	0.063	0.049	0	100	0	21.2
Sailor2-1B	0.075	0.037	0.071	0.043	0	100	0	35.0
Mean	0.077	0.042	0.061	0.036	0	100	0	24.2

its effectiveness and false-positive behavior vary across models. As shown in Table 4, the guard lowers mean ASR from 0.077 without defense to 0.036. It produces the lowest ASR for Llama-3.2-1B, Qwen2.5-1.5B, and Gemma-2-2B, whereas keyword filtering yields lower ASR for Qwen2.5-3B and Sailor2-1B.

Keyword filtering reduces mean ASR to 0.042, but incorrectly blocks all clean look-alike cases, resulting in an FPR of 100%. Spotlighting incurs no blocking false positives and lowers mean ASR to 0.061, although its reduction is smaller than those achieved by the guard and keyword filtering. The guard obtains the lowest mean ASR with a mean FPR of 24.2%, ranging from 0% for Gemma-2-2B to 35% for Llama-3.2-1B and Sailor2-1B.

Table 5 complements the security analysis by reporting benign task utility on the clean test cases. Spotlighting achieves the highest mean BTU at 68.2%, compared with 64.8% without defense, while blocking no clean cases. Keyword filtering reduces mean

Table 5. Benign task utility under the evaluated defense configurations. Values in parentheses denote the number of blocked clean cases.

Agent model	Keyword			
	None	filter	Spotlighting	Guard
Llama-3.2-1B	56.7	54.3 (85)	63.3 (0)	50.7 (285)
Qwen2.5-1.5B	65.8	62.1 (85)	68.9 (0)	62.3 (239)
Gemma-2-2B	65.7	61.5 (85)	70.3 (0)	63.1 (132)
Qwen2.5-3B	65.8	62.3 (85)	69.8 (0)	61.0 (289)
Sailor2-1B	69.9	63.8 (85)	68.7 (0)	47.2 (361)
Mean	64.8	60.8	68.2	56.9

BTU to 60.8%, whereas the three-view guard yields the lowest mean BTU at 56.9%. The utility effect remains model dependent: spotlighting improves BTU for four of the five models, while the guard retains slightly higher BTU than keyword filtering for Qwen2.5-1.5B and Gemma-2-2B but lower BTU for the other three models. The largest decrease under the guard is observed for Sailor2-1B, whose BTU falls from 69.9% to 47.2%. These results show that the guard’s lower residual attack success is accompanied by a greater average loss of benign utility, whereas spotlighting preserves a more favorable utility profile.

Figure 4 examines the contribution of additional language views for Llama-3.2-1B and Qwen2.5-1.5B. The one-view configuration provides no cross-view disagreement signal. Adding the diacritic-free Vietnamese view reduces ASR and produces a nonzero detection rate, while incorporating the English-translated view further lowers ASR and

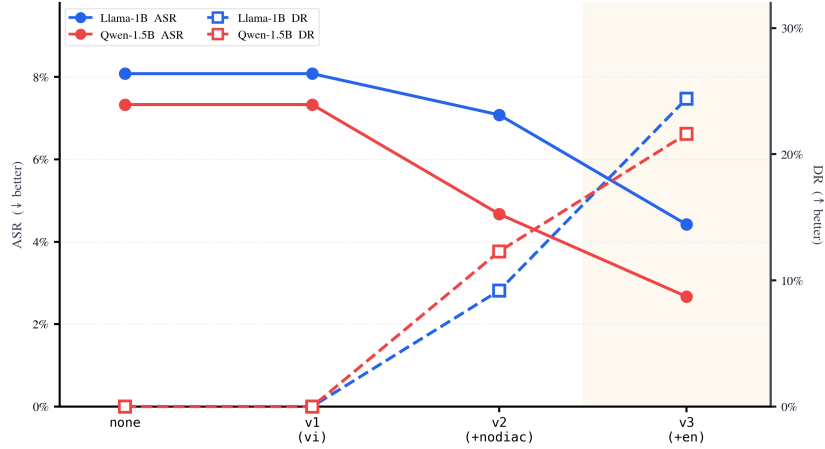


Fig. 4. View ablation for Llama-3.2-1B and Qwen2.5-1.5B. Solid lines report ASR on the left axis, while dashed lines report detection rate on the right axis.

increases detection for both models. This suggests that the translated view provides an additional cross-view disagreement signal beyond Vietnamese normalization alone.

These gains come with additional inference cost. The undefended and spotlighting configurations require one agent pass per case, while keyword filtering requires at most one pass because flagged inputs can be rejected before inference. The full guard requires three passes, one for each language view. Overall, the guard achieves the lowest mean residual ASR and avoids the complete rejection of clean look-alike cases observed with keyword filtering, but introduces model-dependent false positives, a larger reduction in benign utility, and three agent-inference passes per case. Spotlighting preserves higher benign utility and incurs no blocking false positives, but provides a smaller reduction in attack success than the guard.

5 Conclusion

We introduced ViTransInject, a framework for evaluating translation-mediated indirect prompt injection in Vietnamese LLM agents. The framework combines ViTransInject-Bench, a paired evaluation protocol for measuring changes across linguistic representations, and a Cross-View Consistency Guard that compares agent actions across multiple language views.

Our results show that attack success varies across deployment configurations and linguistic representations, with substantial differences across agent models. The proposed guard reduces residual attack success, but also introduces additional inference cost, model-dependent false positives, and benign utility loss. These findings indicate that language preprocessing and translation should be considered when evaluating the security of multilingual agent pipelines.

References

1. An, H., Zhang, J., Du, T., Zhou, C., Li, Q., Lin, T., Ji, S.: IpiGuard: A novel tool dependency graph-based defense against indirect prompt injection in llm agents. In: Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing. pp. 1023–1039 (2025)
2. Chen, S., Piet, J., Sitawarin, C., Wagner, D.: {StruQ}: Defending against prompt injection with structured queries. In: 34th USENIX Security Symposium (USENIX Security 25). pp. 2383–2400 (2025)
3. Debenedetti, E., Zhang, J., Balunovic, M., Beurer-Kellner, L., Fischer, M., Tramèr, F.: Agent-dojos: A dynamic environment to evaluate prompt injection attacks and defenses for llm agents. *Advances in neural information processing systems* **37**, 82895–82920 (2024)
4. Deng, Y., Zhang, W., Pan, S.J., Bing, L.: Multilingual jailbreak challenges in large language models. In: The Twelfth International Conference on Learning Representations (2024), <https://openreview.net/forum?id=vESNKdEMGp>
5. Dou, L., Liu, Q., Zhou, F., Chen, C., Wang, Z., Jin, Z., Liu, Z., Zhu, T., Du, C., Yang, P., Wang, H., Liu, J., Zhao, Y., Feng, X., Mao, X., Yeung, M.T., Pipatanakul, K., Koto, F., Thu, M.S., Kydlíček, H., Liu, Z., Lin, Q., Sripaisarnmongkol, S., Sae-Khow, K., Thongchim, N., Konkaew, T., Borijindargoon, N., Dao, A., Maneegard, M., Artkaew, P., Yong, Z.X., Nguyen, Q., Phatthiyaphaibun, W., Tran, H.H., Zhang, M., Chen, S., Pang, T., Du, C., Wan,

- X., Lu, W., Lin, M.: Sailor2: Sailing in south-east asia with inclusive multilingual llms (2025), <https://arxiv.org/abs/2502.12982>
6. Grattafiori, A., Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., Letman, A., Mathur, A., Schelten, A., Vaughan, A., et al.: The llama 3 herd of models. arXiv preprint arXiv:2407.21783 (2024)
7. Hines, K., Lopez, G., Hall, M., Zarfati, F., Zunger, Y., Kiciman, E.: Defending against indirect prompt injection attacks with spotlighting, 2024. URL: <https://arxiv.org/abs/2403.14720> (2024)
8. Hofman, O., Brokman, J., Rachmil, O., Bose, S., Pahuja, V., Shimizu, T., Starostina, T., Marchisio, K., Goldfarb-Tarrant, S., Vainshtein, R.: Maps: A multilingual benchmark for agent performance and security. In: Findings of the Association for Computational Linguistics: EACL 2026. pp. 821–845 (2026)
9. Jia, F., Wu, T., Qin, X., Squicciarini, A.: The task shield: Enforcing task alignment to defend against indirect prompt injection in llm agents. In: Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). pp. 29680–29697 (2025)
10. Li, R., Chen, M., Hu, C., Chen, H., Xing, W., Han, M.: Gentel-safe: A unified benchmark and shielding framework for defending against prompt injection attacks. arXiv preprint arXiv:2409.19521 (2024)
11. Liu, Y., Jia, Y., Geng, R., Jia, J., Gong, N.Z.: Formalizing and benchmarking prompt injection attacks and defenses. In: 33rd USENIX Security Symposium (USENIX Security 24). pp. 1831–1847 (2024)
12. Qwen, ., Yang, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Li, C., Liu, D., Huang, F., Wei, H., Lin, H., Yang, J., Tu, J., Zhang, J., Yang, J., Yang, J., Zhou, J., Lin, J., Dang, K., Lu, K., Bao, K., Yang, K., Yu, L., Li, M., Xue, M., Zhang, P., Zhu, Q., Men, R., Lin, R., Li, T., Tang, T., Xia, T., Ren, X., Ren, X., Fan, Y., Su, Y., Zhang, Y., Wan, Y., Liu, Y., Cui, Z., Zhang, Z., Qiu, Z.: Qwen2.5 technical report (2025), <https://arxiv.org/abs/2412.15115>
13. Ranaldi, L.: Multilingual retrieval-augmented generation for knowledge-intensive question answering task. In: Findings of the Association for Computational Linguistics: EACL 2026. pp. 697–716 (2026)
14. Team, G., Riviere, M., Pathak, S., Sessa, P.G., Hardin, C., Bhupatiraju, S., Hussenot, L., Mesnard, T., Shahriari, B., Ramé, A., et al.: Gemma 2: Improving open language models at a practical size, 2024. URL <https://arxiv.org/abs/2408.00118> 1(3) (2024)
15. Wendler, C., Veselovsky, V., Monea, G., West, R.: Do llamas work in english? on the latent language of multilingual transformers. In: Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). pp. 15366–15394 (2024)
16. Wu, Y., Schuster, M., Chen, Z., Le, Q.V., et al.: Google’s neural machine translation system: Bridging the gap between human and machine translation. arXiv preprint arXiv:1609.08144 (2016)
17. Yi, J., Xie, Y., Zhu, B., Kiciman, E., Sun, G., Xie, X., Wu, F.: Benchmarking and defending against indirect prompt injection attacks on large language models. In: Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 1. pp. 1809–1820 (2025)
18. Yong, Z.X., Menghini, C., Bach, S.: Low-resource languages jailbreak GPT-4. In: Socially Responsible Language Modelling Research (2023), <https://openreview.net/forum?id=pn83r8V2sv>
19. Zhan, Q., Liang, Z., Ying, Z., Kang, D.: Injecagent: Benchmarking indirect prompt injections in tool-integrated large language model agents. In: Findings of the Association for Computational Linguistics: ACL 2024. pp. 10471–10506 (2024)

20. Zhu, K., Yang, X., Wang, J., Guo, W., Wang, W.Y.: MELON: Provable defense against indirect prompt injection attacks in AI agents. In: Singh, A., Fazel, M., Hsu, D., Lacoste-Julien, S., Berkenkamp, F., Maharaj, T., Wagstaff, K., Zhu, J. (eds.) Proceedings of the 42nd International Conference on Machine Learning. Proceedings of Machine Learning Research, vol. 267, pp. 80310–80329. PMLR (13–19 Jul 2025), <https://proceedings.mlr.press/v267/zhu25z.html>